Power-Grid SDK Plugin Building Guide v0.1

Roccat Testlab*

Version History
August 14, 2015
- initial version

***contact**: support@roccat.org

Contents

Introduction 1
1 Power-Grid Architecture 2
2 Plugin Architecture 3
3 Getting Started: plugin.json 4
3.1 Providing Pre-Made Grids . . . 5
3.2 File Hierarchy . . . 5
4 Adding Javascript 6
5 Custom DLLs 8
5.1 Bridging to Another Application . . . 8

Introduction

This document explains how to create plugins for Power-Grid. Plugins can be created from JSON files, Javascript files and even compiled DLLs, each building upon each other.
The SDK was designed to give you everything you need to create whatever control or grid you want, from simple reskinning to full-blown application-specific grids.

Please read this document carefully, since using the SDK incorrectly may cause unexpected (and unneccessary) Power-Grid slowdowns when using your plugin.

Power-Grid and this SDK are constantly improved and enhanced, so in the future you will find more and more features at your disposal for writing really cool plugins.

We tried to keep the SDK slim and functional to suit all needs that may arise. If however, you find that the SDK is missing a key feature, do not hesitate to contact us at: support@roccat.org.

1. Power-Grid Architecture

Power-Grid is a PC-remote application consisting of two parts: host and client.
The installed host (in the following sections called ”launcher”) gathers information on the host system and sends input to applications on the host system.
The client is a thin-client which connects via UDP/TCP with the host application and is able to recieve and send information.

In general all logic is done by the host, even rendering composite images. The client only recieves a list of images it has to display along with minor layout information.

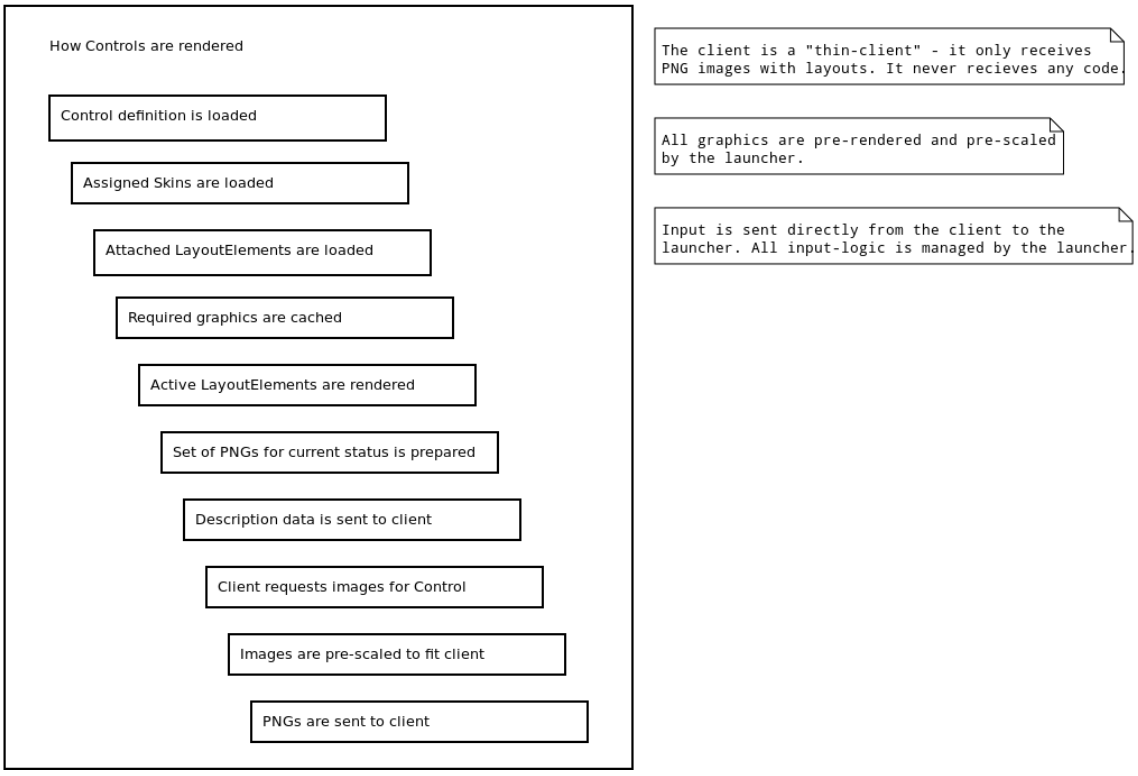


Figure 1. Power-Grid Control Rendering Scheme

In the following sections we assume you are familiar with Power-Grid in general. You should have a basic understanding on what a control and a grid is. Preferably you have also already experimented with the basic GUI-based grid builder that comes with the software.

2. Plugin Architecture

Each Power-Grid plugin is placed in its own folder inside the plugins directory in the userdata folder.
Every plugin requires a 'plugin.json' file in the root of it's plugin folder. In this file all controls are described and configured.

Further a '*.ctrlset.json' file is required for plugins that offer new controls to the user - which will most likely apply to all plugins. If you really want to avoid single controls of your plugin to be listed for user - if you are offering a complete grid - then you can leave out the '*.ctrlset.json' and instead only supply a '*.grids.json' file. There is however no security lockout available to prevent users from supplying their own '*.ctrlset.json'.
The advanced functions can be written in Javascript or supplied through native code DLLs, depending on your needs.

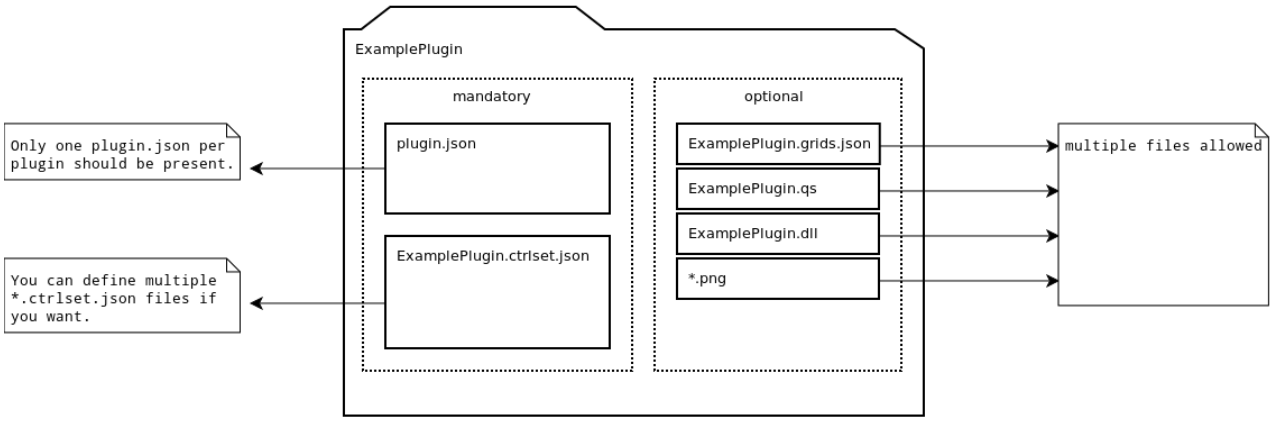


Figure 2. Plugin Architecture

We have chosen Javascript as primary scripting language because it's very well documented and many users are already familiar with it. A custom script language might offer more specialized functions, but it has to be learned from scratch. If you are concerned about performance you are free to use DLLs whenever you please.
The Javascript code will bridge between your Power-Grid layouts and the DLL's functions. You cannot change your controls directly from the DLL, you always have to go through the Javascript first.
See the following image for clarification:

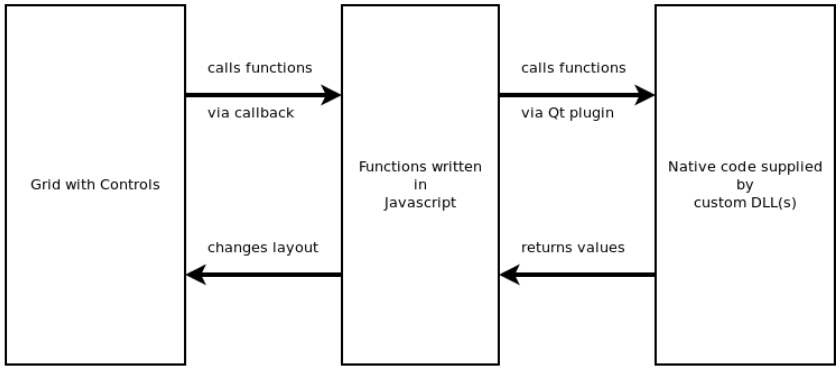


Figure 3. From Power-Grid to DLL via Javascript

3. Getting Started: plugin.json

When writing a plugin for Power-Grid, the first thing you need to create is a plugin.json file.
The bare minimum for a single control should look like this:

```
{
    Name:"MyPlugin",
    CtrlSets:["MyPlugin.ctrlset.json"],
    Skins:[
        {
            Name:"MySkin",
            Layouts:[
                {
                    Name:"MyLayoutElement",
                    Type:"Gfx",
                    Gfx:"MyGfx",
                    Left: "0",
                    Top: "0",
                    Height: "160",
                    Width: "160"
                }
            ] // MySkin::Layouts
        } // MySkin
    ], // Skins
    Gfxs:[
        {
            Name:"MyGfx",
            Rsc:"MyGfx.png",
            Tiling:"None",
        }
    ], // Gfxs
    Controls:[
        {
            Name:"MyControl",
            Skins:["MySkin"],
            Config:{
                Description:"My first Plugin Control",
            },
            Width:1,
            Height:1,
        },
    ]
}
```

Do not forget to create a matching MyPlugin.ctrlset.json file as well:

```
{
    Type:"ControlSet",
    Name:"[Plugin] MyPlugin",
    ExpandCategories:["*"],
    Elements:[
        {
            Name:"MyControl",
            Control:"MyPlugin:MyControl",
            Category:"Input"
        }
    ]
}
```

Placing these two files in a directory inside your plugins folder should make your plugin show up in Power-Grid. (Remember to restart Power-Grid each time you change a plugin or add a new one manually).

This should get you started with the basic structure of how controls are defined in plugins. Pay attention to each key-value as 'Width' and 'Height' are not always the same depending on their placement. In a LayoutElement they are considered to be given in pixels, whereas for Controls they refer to grid-cells.

3.1 Providing Pre-Made Grids

If you want to supply pre-made grids along with your plugin you can define grids with a grids.json file. The syntax is quite simple, just take a look at this example:

```
{
    // Namespace
    Name:"[MyPlugin]",
    Grids: [
                {
                    Icon:"my_grid_icon.png|0.5||CC: 0,-28",
                    Name:"MyFirstGrid",
                    Title:"MyGRID",
                    Elements:[
                        {
                            Control:"MyPlugin:MyControl",
                            Name:"",
                            ResetName:"MyControl",
                            SkinIdx:0,
                            Left:0,
                            Top:0
                        }
                        {
                            Control:"MyPlugin:MyControl",
                            Name:"",
                            ResetName:"MyControl",
                            SkinIdx:0,
                            Left:1,
                            Top:0
                        }
                    ], // Elements
                } // MyFirstGrid
    ] // Grids
}
```

This will offer the user the grid "MyFirstGrid" in his grid list. It will have the control "MyControl" twice on positions (0,0) and (1,0), using the image my_grid_icon.png in the tab bar with the text "MyGRID".
You can define multiple grids here, but try to avoid overlapping controls as this may lead to undefined behaviour when touch events happen.

3.2 File Hierarchy

Since this SDK does not provide an IDE currently, this diagram is provided for your convenience.

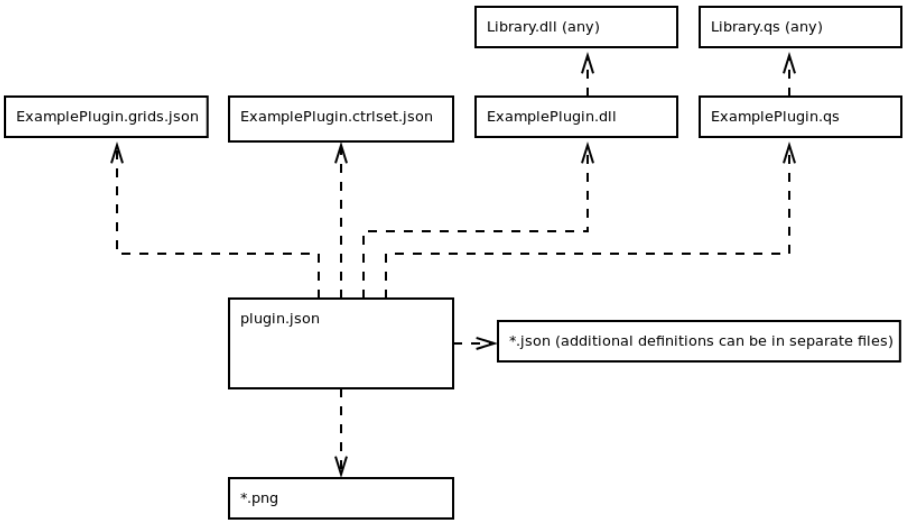


Figure 4. Plugin File Hierarchy

4. Adding Javascript

While defining new controls is nice, you will want to add more functionality to your controls than offered by default Power-Grid buttons.

To do this, you need to create a Javascript file MyPlugin.qs and load it accordingly:

```
...
Scripts:[
        {
            Name:"MyPluginScript",
            Functions:[
                "MyOnMouseClick",
            ],
            // This is the filename for the script
            Rsc:"MyPlugin.qs"
        }
    ],
...
```

The Javascript file is now loaded by your plugin.
Note: you need to define all methods that are to be executed on certain events up-front. This is why there is a Functions-array listed. You only need to list those functions that you want to be called by Power-Grid on specific events, subfunctions such as helper methods can safely left out of this list.

Suppose you want a counter to count up from 0 to 9 each time you tap it, you would define the following MyPlugin.qs file:

```
var my_global_counter = 0;
function MyOnMouseClick(p_xSkin) {
    my_global_counter = (my_global_counter+1)%10;
    var mytext = p_xSkin.FindLayout("MyText");
    mytext.setText(my_global_counter);
}
```

You can use any valid Javascript here, but be aware that not all extensions that you are familiar with will be available. You are free, however, to add the required code yourself.

To change the text in your control you should also add it to the plugin.json first:

```
...
        {
            Name:"MySkin",
            OnMouseClick:"MyOnMouseClick",
            Layouts:[
                {
                    Name:"MyLayoutElement",
                    Type:"Gfx",
                    Gfx:"MyGfx",
                    Left: "0",
                    Top: "0",
                    Height: "160",
                    Width: "160"
                },
                {
                    Name:"MyText",
                    Type:"Text",
                    Text:"-",
                    Font:"EuroD,16",
                    Color:"Blue",
                    Left: "0",
                    Top: "0",
                    Height: "160",
                    Width: "160"
                }
            ] // MySkin::Layouts
        } // MySkin
    ], // Skins
```

...

This should make the text appear on top of the control. Notice the the callback for OnMouseClick in the MySkin block, this is where you can link your function to be executed on each tap of the control.

You should now be able to add the control from your plugin to a grid and use it to count from 0 to 9 on each tap.

There are many possibilities to add functionality to your controls, as well as many styling options for skins. Please refer to the API documents supplied with this documentation and also take a look at the supplied TemplateControl, which should contain an example of every key-value you can use.

We will constantly improve this documentation with more examples. Please also take a look at the forum and if you are stuck just send an email to support@roccat.org.

5. Custom DLLs

If you are keen to have total freedom on what event to start when pressing a control, or what information to be displayed on your grid, then you might need to supply a custom DLL.

Power-Grid is written using the Qt Framework, and it supports basic Qt plugins.

For creating and compiling a plugin DLL, the following toolchain is required:

Qt Libraries version 4.*.* http://download.qt.io/archive/qt/4.8/4.8.6/

MSVC 2010 SP1 Toolchain http://go.microsoft.com/?linkid=9709969 (you can use the 2010 Express version, which is available for free)

QtCreator http://www.qt.io/download-open-source/ (recommended IDE)

or

Qt Visual Studio Add-in http://www.qt.io/download-open-source/ (does NOT WORK with VSExpress)

Once you have setup your basic structure for building a DLL with Qt support, you are pretty much free to do anything you like in the DLL. But know that the code is executed inside the Power-Grid process and therefore should not be very resource intense if it's called often. If you want to do intense calculations consider using an additional thread.

Please refer to the sample DLL sourcecode that comes with the SDK for more information.

5.1 Bridging to Another Application

If you want to link your application to your plugin you need to supply the code for data exchange yourself. We currently do not offer a generalized method for inter-process-communication.
Possible methods would be to use FIFOs, Sockets, WindowMessages, the Registry or just frequently updated files. Depending on your application other methods might be possible too.
We suggest, however, to consider portability to other OS's (OSX/Linux/SteamOS) should your application be cross platform. Although we are currently only offering Power-Grid for Microsoft Windows this might change in the future.

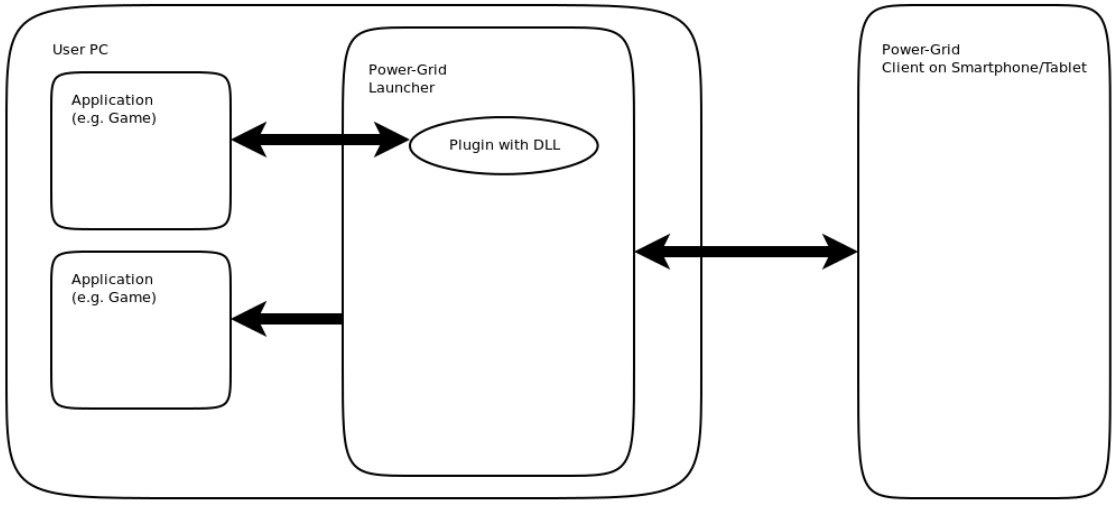


Figure 5. DLL schema